SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME:Software Testing. **COURSE CODE:**

CSA3711 COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3

3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5
4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3

8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5
10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Solutions:

1. Errors in Equivalence Partitioning and Boundary Value Analysis

The tester has incorrectly designed test cases for the online shopping system by mixing valid and invalid inputs in the same equivalence class. Equivalence Class Partitioning (ECP) divides inputs into groups that are expected to behave similarly. Each class should contain either valid or invalid inputs according to the system requirements. For example, if the permitted product quantity is 1–100, valid inputs belong to the range 1–100, while values below 1 and above 100 belong to invalid classes.

The root cause of the mistake is the incorrect identification of input ranges and the failure to consider boundary conditions. Boundary Value Analysis (BVA) focuses on values at and near the limits because

errors frequently occur at these points. For the quantity range 1–100, the important boundary test values are 0, 1, 2, 99, 100, and 101. Testing only ordinary values such as 25 or 50 may fail to reveal errors in minimum and maximum quantity validation.

The corrected strategy should define separate equivalence classes for quantities below 1, between 1 and 100, and above 100. Test cases should include 0, 1, 2, 50, 99, 100, and 101, with expected results specified for each input. Similar partitions should be created for product prices, discount percentages, and customer details. This approach improves defect detection by systematically testing valid inputs, invalid inputs, and critical boundaries instead of relying on incomplete or incorrectly classified test data.

2. Random Testing in a Banking Application

Random testing selects inputs without following a systematic test design method. Although it may discover unexpected defects, it does not guarantee adequate coverage of transaction limits, validation rules, or critical execution scenarios. The QA engineer's claim of high coverage is incorrect if it is based only on the number of random inputs tested. A large number of random tests can repeatedly exercise normal transactions while missing rare but important boundary conditions.

The main cause of missing defects is the absence of requirements-based test selection and measurable coverage criteria. For example, a banking application may permit transfers between ₹100 and ₹50,000. Randomly selecting amounts such as ₹500, ₹2,000, and ₹5,000 does not adequately test the minimum and maximum limits. Defects involving ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001 may remain undetected. Random testing may also overlook insufficient funds, invalid account details, duplicate transactions, and unauthorized operations.

The improved approach should combine Equivalence Class Partitioning, Boundary Value Analysis, Decision Table Testing, and State Transition Testing. ECP separates valid and invalid transaction inputs, BVA checks values near transaction limits, decision tables test combinations of conditions, and state transition testing verifies transaction states such as pending, successful, and failed. Random testing can supplement these techniques to discover unexpected inputs. This structured strategy provides more measurable coverage and improves banking reliability by targeting critical conditions and transaction failures.

3. Incomplete Control Flow Graph and Path Testing

A Control Flow Graph (CFG) represents the execution flow of a program through nodes and directed edges. If a developer omits decision nodes or loops, the graph will not represent all possible execution paths. For

example, an if-else statement must include both outcomes, and a loop must represent its entry condition, body, and exit path. An incomplete CFG can cause testers to overlook error-handling branches and repeated execution scenarios.

The root causes of inadequate coverage include misunderstanding control structures, ignoring loop conditions, and creating test cases without examining the complete program logic. Independent paths are important because they represent distinct routes through the program. Cyclomatic complexity can estimate the number of linearly independent paths using $V(G) = E - N + 2P$, where E is the number of edges, N is the number of nodes, and P is the number of connected components. An incorrect CFG produces an unreliable complexity estimate and an incomplete path-testing plan.

The correction is to reconstruct the CFG with every statement, decision outcome, loop entry, and loop exit represented. Test cases should cover each independent path, both true and false outcomes of decisions, and loop execution with zero, one, and multiple iterations. For example, a withdrawal module should test successful withdrawal, invalid PIN, insufficient balance, and exceeded withdrawal limits. This improves branch and path coverage, helps reveal hidden logical defects, and ensures that important program routes are not excluded from testing.

4. Limitations of 100% Statement Coverage

Statement coverage measures whether every executable statement has been executed at least once. Although the testing team achieved 100% statement coverage, this does not prove that every decision outcome or logical condition was tested. A test may execute all statements through only one branch of a complex decision, leaving alternative branches untested. Therefore, the reported coverage is insufficient to establish the correctness of the module.

The root cause is the assumption that executing every statement guarantees complete testing. Consider if (balance >= amount), which determines whether a withdrawal is permitted. A test using sufficient balance may execute the withdrawal statement, but without testing insufficient balance, the rejection branch may remain untested. Similarly, complex Boolean expressions can contain defects even when their associated statements execute. Statement coverage does not independently verify true and false outcomes of decisions.

The testing strategy should be improved by adding branch coverage, condition coverage, and path testing. Branch coverage requires every decision outcome to execute, while condition coverage checks whether each individual Boolean condition becomes true and false. Path testing examines important independent execution routes. For the withdrawal example, tests should include sufficient balance, insufficient balance,

valid PIN, and invalid PIN. These techniques provide stronger evidence of test adequacy and increase the likelihood of detecting incorrect conditions, missing error handling, and logical defects.

5. Incorrect Equivalence Classes for Age Validation

The tester has defined only two equivalence classes, negative numbers and positive numbers, for an age field that accepts values from 0 to 100. This classification is incorrect because it groups valid and invalid positive values together. For example, age 50 is valid, but age 101 is invalid, even though both are positive. A correct equivalence partition must be based on the application's actual validation requirements rather than simply on whether a number is positive or negative.

The main problem is that the tester ignored the permitted age range and failed to distinguish values below the minimum, values within the valid range, and values above the maximum. Boundary Value Analysis is also missing, so defects at ages 0 and 100 may remain undetected. Other potentially invalid inputs, such as empty fields, non-numeric characters, and decimal values, should be considered according to the system's input requirements.

The corrected equivalence classes are: invalid ages below 0, valid ages from 0 to 100 inclusive, and invalid ages above 100. Boundary tests should include -1, 0, 1, 99, 100, and 101. Additional tests should cover empty input, alphabetic input, and decimal values if the application accepts raw user-entered data. This combination of ECP and BVA provides a clear classification of valid and invalid inputs and improves the reliability of age validation.

6. Incorrect Condition Coverage for a Complex Boolean Expression

Condition coverage verifies that each individual Boolean condition evaluates to both true and false at least once. For a decision such as `if (A && B)`, both A and B must independently take true and false values across the test suite. A developer cannot claim full condition coverage merely because the overall decision has evaluated to both true and false. The individual conditions must be examined separately.

The root cause of inadequate testing may be the use of test cases that do not independently exercise each condition. For example, testing only A = true, B = true and A = false, B = false gives both conditions true and false across the suite, but may not reveal how each condition affects the overall decision independently. Short-circuit evaluation also matters because when A is false, B may not be evaluated in the expression `A && B`.

A better test set for A && B is (true, true), (true, false), and (false, true), with expected decision outcomes of true, false, and false, respectively. These cases exercise both outcomes of each condition and demonstrate the independent effect of changing a condition while holding the other fixed for relevant pairs. For safety-critical or complex logic, Modified Condition/Decision Coverage (MC/DC) can provide stronger assurance that each condition independently affects the decision outcome. This improves logical defect detection beyond testing only the final decision result.

7. Incomplete Decision Table for a Discount System

Decision Table Testing is suitable for a discount system because the final price depends on combinations of conditions, such as premium membership and coupon application. If the testing team omits combinations, certain discount rules may not be executed. For example, testing premium members without coupons and ordinary customers with coupons does not establish how the system behaves when a premium member also applies a coupon.

The root cause is an incomplete decision table that does not list every meaningful combination of conditions and their corresponding actions. Assuming two Boolean conditions—Premium Member and Coupon Applied—there are four possible combinations. If one combination is missing, the implementation may incorrectly combine discounts, reject a valid coupon, or apply an unintended discount. The table must also define whether discounts can be combined and what happens when a coupon is invalid or expired.

A revised decision table should include all four combinations: (1) premium member, coupon applied; (2) premium member, no coupon; (3) non-premium member, coupon applied; and (4) non-premium member, no coupon. Each row should specify the expected discount and final price according to the business rules. Additional cases should cover invalid coupons, expired coupons, minimum purchase requirements, and discount caps. This systematic approach ensures that important condition combinations are tested and reduces the risk of incorrect price calculations in production.

8. Inadequate Loop Testing Strategy

The tester has checked only the first and last iterations of a loop, using $n = 1$ and $n = max$. This is insufficient because defects may occur when the loop executes zero times, when it executes once, at intermediate iterations, or when it exceeds the permitted limit. Testing only the first and last iterations can miss incorrect initialization, termination conditions, off-by-one errors, and failures during repeated execution.

The root cause is an incomplete understanding of loop boundary conditions and an assumption that testing typical or maximum iteration counts is enough. For a loop that permits up to 10 iterations, useful test values include $n = 0$, $n = 1$, $n = 2$, $n = 9$, $n = 10$, and $n = 11$, where applicable. The test at $n = 11$ checks how the system handles an input beyond the permitted maximum. For nested loops, defects may also arise from interactions between inner and outer loop counters.

The corrected strategy should test zero iterations, one iteration, two iterations, a typical number, the maximum number, and one beyond the maximum. Simple loops should be tested independently before nested loops are examined. For nested loops, begin by testing each loop separately, then test combinations of boundary values and selected iteration counts for the inner and outer loops. These tests help identify incorrect loop termination, repeated processing, and boundary defects while improving confidence in execution flow.

9. Error Guessing in a Safety-Critical Braking System

Error Guessing relies on a tester's experience to predict likely defects, such as incorrect sensor readings, invalid braking commands, or unexpected input values. Although it can identify useful edge cases, using it as the sole testing technique for a safety-critical braking system is inadequate. Ad-hoc testing does not guarantee systematic coverage of input boundaries, operating states, or hazardous transitions, and its results depend heavily on the tester's knowledge.

The root cause is the failure to use structured, requirements-based techniques alongside experience-based testing. A braking system may have different operating states, such as normal braking, emergency braking, ABS activation, and fault mode. Incorrect sensor values, delays, or transitions between these states can have serious consequences. Error Guessing alone may miss subtle defects involving threshold values, state changes, sensor failures, or recovery behavior.

The improved strategy should combine Boundary Value Analysis with State Transition Testing and retain Error Guessing as a supplementary technique. BVA should test sensor readings and braking thresholds at, below, and above defined limits. State Transition Testing should verify legal and illegal transitions between normal operation, emergency braking, and fault states. Additional tests should cover sensor failures, delayed responses, and recovery from faults. This structured approach improves coverage of critical conditions and strengthens safety assurance, although formal safety analysis and applicable automotive standards are also necessary.

10. Missing Else Branches in a Control Flow Graph

A Control Flow Graph that omits else branches from nested if statements does not accurately represent the program's execution. Each decision must include both true and false outcomes, even when one outcome contains only error handling or transfers control to another part of the program. Missing these branches can cause the tester to overlook invalid-input handling, failed validations, and alternative execution paths while incorrectly assuming that the CFG is complete.

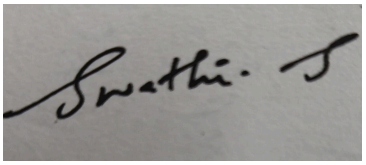
The root causes include incomplete code analysis, treating error-handling statements as unimportant, and designing test cases only for successful execution. For example, a transaction module may check account validity, sufficient balance, and transaction limits through nested decisions. If the graph represents only the successful outcomes, tests may execute every main statement while never testing invalid accounts, insufficient funds, or exceeded limits. Consequently, statement coverage may reach 100% while branch coverage remains incomplete.

The CFG should be reconstructed to include every nested decision, both true and false edges, error-handling nodes, and all relevant exit paths. Independent paths should cover successful processing, invalid account details, insufficient balance, exceeded transaction limits, and combinations of failed conditions. Test cases must demonstrate execution of every branch and verify the expected result for each path. This correction improves branch adequacy, exposes missing or incorrect error handling, and provides stronger confidence that both normal and exceptional execution paths behave correctly.

Code of Conduct:

I (192421111) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand